Al Rafiq Shopping Centre

CTO Deployment Readiness Audit

Deployment Readiness Score

58 / 100

Conditional GO for soft launch / staging / beta.

NO-GO for a revenue-bearing storefront until P0 + P1 items are closed.

Prepared by: CTO · DevOps Architect · Cloud Solutions Architect · Security Engineer

Audit basis: direct inspection of the source code (cited by file:line)

Date: 2026-05

Auditor	CTO / DevOps Architect / Cloud Solutions Architect / Security Engineer
Audit basis	Direct inspection of source (cited by file:line)
Audit date	2026-05
Codebase scope	Frontend (React/Vite) + Backend (Node/Express + JSON store)

Executive Summary

Al Rafiq Shopping Centre is a **feature-complete demo / soft-launch grade** single-service web application: React/Vite frontend + Node/Express backend served from one URL. Build is green; routing, RBAC, admin tooling and AI features are working. However, three structural items make it **not yet production-grade for a real multi-customer store**:

1. **No real database** - persistence is a single JSON file (`server/data/db.json`, `server/src/store.js:10`). Single-writer, no concurrency control, no indexes.
2. **Storefront data lives in browser localStorage** for every visitor (`src/store/index.js:60-68`). Until migrated to the server, products / orders you add in admin are not visible to other customers.
3. **Several security defaults must be overridden** before launch (default JWT secret in code, default seeded passwords, wide-open CORS, no rate limiting, no security headers).

Deployment Readiness Score: 58 / 100 (soft-launch / staging ready; not yet ready for paying customers at scale).

Go / No-Go: Conditional GO for soft-launch / demo / staging. **NO-GO for a real revenue-bearing storefront** until items P0 and P1 in Phase 10 are completed.

PHASE 1 - Project Analysis

Application Overview

Item	Value	Evidence
Project type	Premium eCommerce (storefront + admin CMS)	`package.json` description
Frontend framework	React 18.3.1	`package.json`
Frontend tooling	Vite 5.3.1, Tailwind 3.4.4, Vite-PWA 0.20	`vite.config.js`, `tailwind.config.js`
Routing	react-router-dom 6.23.1 (lazy routes)	`src/App.jsx`
State	Redux Toolkit 2.2.5 + react-redux 9.1.2 + localStorage persistence	`src/store/index.js`

Item	Value	Evidence
UI / motion	lucide-react 0.400, framer-motion 11, recharts 2.12, swiper 11	`package.json`
SEO / meta	react-helmet-async 2.0.5; sitemap.xml + robots.txt + manifest.webmanifest	`public/`
Backend framework	Express 4.19.2 on Node.js 20 (ESM)	`server/package.json`
Auth	JSON Web Tokens (jsonwebtoken 9.0.2) + bcryptjs 2.4	`server/src/auth.js`
File processing	multer 1.4.5-lts, sharp 0.33.4	`server/src/routes/uploads.routes.js`
Exports	exceljs 4.4 (xlsx) + pdfkit 0.15 (PDF)	`server/src/util/exporters.js`
Smart product import	cheerio 1.0 + native fetch (SSRF-guarded)	`server/src/util/scrape.js`
AI engine	Stub by default; live Claude via Anthropic Messages API (fetch) gated by `USE_REAL_AI` + `CLAUDE_API_KEY`	`server/src/services/ai.js`
Data store	**JSON file (`server/data/db.json`)** + in-memory cache	`server/src/store.js`
Storefront data	**Browser localStorage** (per visitor)	`src/store/index.js:60-68`
Payments	COD, Bank Transfer (no gateway integrated)	`src/pages/Checkout.jsx`
Email	mailto: scheme + console; EmailJS hooks commented for future	`src/utils/notify.js`
Storage (uploads)	Local filesystem (`server/data/uploads/YYYY/MM/`)	`server/src/util/images.js`
Third-party SDKs	firebase 10.12 (declared, not actively used in core flows)	`package.json`

Build Analysis

Check	Status	Evidence
Frontend production build	PASS - bundles cleanly (esbuild full-tree parse)	repeated `npm run build` -> dist/
Bundle size	~600 KB raw (unminified), code-split via Vite `manualChunks` (react, redux, firebase, recharts, motion)	`vite.config.js`, esbuild output
Server syntax	PASS - `node --check` on 19 files	repeated audit
Local import resolution	PASS (resolver script ran clean after the 2 `routes/ ../../../` path bugs we already fixed)	audit script

Check	Status	Evidence
Unused imports	PASS (zero across `src`) after one cleanup	static check
Vulnerable packages	Not run live (`npm audit` exceeds sandbox); declared versions are recent and unpinned (`^`)	`package.json`
Missing env vars	`JWT_SECRET`, `SITE_URL` not required (defaults exist) - **defaults must NOT be used in production**	`server/src/auth.js:7`, `server/src/util/seo.js:67`
Build commands	`npm run setup` (one install does both), `npm run build`, `npm start`, `npm run dev:all`	`package.json`

Fixes required before launch (build/config): ship `.env` with strong `JWT_SECRET`, set `SITE_URL`, pin all `^` dependency ranges before production release, run `npm audit fix`.

PHASE 2 - Infrastructure Audit

Frontend hosting

This is **not a static-only frontend** - the backend serves the built site, so "frontend-only" hosts that don't run Node (pure Cloudflare Pages / Netlify static / S3 + CloudFront) are **inappropriate** for the current architecture.

Option	Verdict	Reason
Vercel (serverless)	Not recommended	Express w/ multer + sharp + file storage doesn't fit short serverless functions
Netlify (static)	Not recommended	Same as above; backend cannot run there
Cloudflare Pages	Only if split	Static front + separate worker/API; current build is one service
AWS S3 + CloudFront	Only if split	As above
Azure Static Web Apps	Only if split	As above
DigitalOcean App / Render / Railway / Fly	RECOMMENDED	Run the single Node service that serves both site + API

Backend hosting

Tier	Recommendation	Why
Shared hosting	NO (unless cPanel "Setup Node.js App" present)	Backend needs Node + sharp + filesystem
cPanel + Node.js Selector	YES for testing/SMB launch	Works; manual build/upload

Tier	Recommendation	Why
VPS (DigitalOcean Droplet, Linode, Hetzner)	YES for scale stage	Full control, predictable cost
Cloud PaaS (Render / Railway / Fly)	YES for fast launch	Built-in TLS, deploys-from-Git, persistent disks available
Docker / Kubernetes	Only at >10k DAU	Overkill for current load

Required servers / specs

Stage	Topology	CPU	RAM	Storage	Notes
Soft launch	1 server (single Node service serves site + API)	1 vCPU	1 GB	10 GB SSD	Free Render/Railway / $5 VPS / cPanel basic
Growth (1-10k users)	1 web + 1 managed DB	2 vCPU	4 GB	50 GB SSD + 10 GB DB	Add CDN for /api/media
Scale (10-100k users)	2 web behind load balancer + managed DB primary + read replica + Redis	4 vCPU each	8 GB each	200 GB + DB + S3 for images	Object storage (S3/R2) for uploads
Enterprise (100k+)	Auto-scaling web tier (k8s) + clustered DB + CDN + edge cache + dedicated workers	4-8 vCPU each, N>=4	16 GB each	S3/R2 (TB)	Background queue (BullMQ/SQS) for AI + heavy reports

For the current architecture **one server is sufficient** for the first ~2,000 monthly active users.

PHASE 3 - Database Analysis

Current technology

No traditional database. Persistence is a single JSON file with an in-memory cache (`server/src/store.js`).

Aspect	Current	Risk
Engine	None - flat JSON file	High - no transactions, no isolation
Concurrency	Single Node process, full file rewrite on every change	Critical above ~5 writes/sec
Indexes	None (Array.filter/find)	Linear scans; fine until ~10k rows
Relationships	Implicit, by id	Manual joins in code

Aspect	Current	Risk
Schema migrations	None	Hand-coded `if (!db.xxx) db.xxx = []` migrations in `index.js`
Backups	Whatever the host disk does	Critical - no point-in-time recovery

Verdict: acceptable for an internal MVP / demo / staging environment.

Unacceptable for a production storefront the moment two admins edit at once or you cross ~hundreds of write ops/min.

Database audit findings

Finding	Severity	Notes
SQL injection	N/A	No SQL
N+1 queries	N/A	No queries, in-memory arrays
Slow queries	LOW today	O(n) scans; will degrade past 5-10k products/orders
Missing indexes	HIGH (architectural)	A real DB will need indexes on slug, sku, orderId, customer email, ticketId
Race conditions	HIGH	Two writes can clobber each other (last-write-wins on full-file overwrite)
Backup / DR	CRITICAL	None built-in

Recommended migration

Option	Best for	Approx. monthly cost (starter)	Notes
Supabase (PostgreSQL + Auth + Storage + Realtime)	RECOMMENDED for this app	Free tier; Pro ~$25	Replaces JSON store, file storage AND auth in one
Neon (serverless Postgres)	Postgres-only	Free tier; ~$19	Great branching/Git workflow
AWS RDS Postgres (db.t4g.micro)	Enterprise/AWS shop	~$15-25	More ops overhead
DigitalOcean Managed Postgres	Simple, predictable	$15	Good for VPS deployments
MongoDB Atlas	Document model	Free M0; M10 ~$57	Easier mapping from current JSON shapes

Recommendation: Supabase (PostgreSQL). It cleanly replaces the JSON store, gives row-level security that mirrors current RBAC, has managed backups, and bundles object storage that can replace `server/data/uploads/`.

PHASE 4 - Storage Requirements

Current

- Uploaded product images: `server/data/uploads/YYYY/MM/<uuid>.webp` (+ thumb).
- `db.json`: ~100 KB - ~5 MB depending on catalog growth.

Recommendation

Asset class	Now	Recommended at scale
Product images / thumbnails	Local disk	**Cloudflare R2** (no egress fees) or AWS S3, both fronted by CDN
Documents (invoices, exports)	Generated on-demand	Stream from server or store in R2/S3 with signed URLs
Audit log archive	In-memory JSON (5000 cap)	Append to managed Postgres + cold-store rolled records in R2
Backups	None	Daily DB snapshot + R2 lifecycle to deep-storage

Storage growth estimate

Scale	Catalog	Avg images / SKU	Image storage
Soft launch	100 SKUs	4	~120 MB
1k users	500 SKUs	5	~750 MB
10k users	2k SKUs	6	~3.5 GB
100k users	10k SKUs	7	~20 GB

Recommendation: Cloudflare R2 ($0.015/GB/mo, no egress).

PHASE 5 - Security Audit

Findings (severity rated; evidence cited)

#	Finding	Severity	Evidence	Remediation
S1	**Default JWT_SECRET hard-coded as fallback** (`'al-rafiq-dev-secret-change-me'`)	**CRITICAL**	`server/src/auth.js:7`	Refuse to start if `JWT_SECRET` is unset or matches default in production. Set a 64-char random secret in env.

#	Finding	Severity	Evidence	Remediation
S2	**Seeded plaintext passwords** ('alrafiq123', 'products123', 'support123', 'accounts123', 'changeme123') hashed and stored on first boot	**CRITICAL** for prod	`server/src/seed.js:10-13`, `server/src/routes/team.routes.js:27`	Force password reset on first login; remove default passwords or require ADMIN_EMAIL/ADMIN_PASSWORD env at seed time.
S3	**CORS allows all origins** (`app.use(cors())` with no config)	**HIGH**	`server/src/index.js:22`	`cors({ origin: process.env.FRONTEND_URL, credentials: true })`.
S4	**No rate limiting** anywhere (login, AI, import, exports) - brute-force / DoS exposure	**HIGH**	`server/src/index.js` (no `express-rate-limit`)	Add `express-rate-limit`: 10/min on `/api/auth/login`, 60/min global, stricter on `/api/import` and `/api/ai/generate`.
S5	**No security headers** (CSP, X-Frame-Options, X-Content-Type-Options, HSTS, Referrer-Policy)	**HIGH**	`server/src/index.js` (no `helmet`)	`app.use(helmet({ contentSecurityPolicy: ... }))`.
S6	**bcrypt cost factor 8**	MEDIUM	`server/src/seed.js:28`, `server/src/routes/team.routes.js:27,58`	Raise to 10-12. Cost 8 is ~5x faster to brute-force than the OWASP 2024 recommendation.
S7	**Body size limit 20 MB** (intended for base64 images)	MEDIUM	`server/src/index.js:23`	Reduce JSON body to 1 MB; keep multer's 12 MB only on the multipart upload route.
S8	**Audit log silently truncated at 5000 entries**	MEDIUM (compliance)	`server/src/audit.js:27`	Rotate to a real append-only sink (DB table) - keep N>=10k recent + cold archive in R2.
S9	`/api/media` **served as static, publicly readable**	LOW (intentional)	`server/src/index.js:21`	OK for product images (need to be public). Consider signed URLs for any "private" assets later.

#	Finding	Severity	Evidence	Remediation
S10	**No CSRF tokens**	LOW	JWT in Authorization header (bearer) is not auto-attached by the browser, so classic CSRF doesn't apply. Re-evaluate if you switch to cookie-based sessions.	Document; revisit on session-change.
S11	**JWT stored in localStorage**	LOW-MEDIUM	`src/api/client.js`	Susceptible to XSS exfiltration if any XSS lands. No `dangerouslySetInnerHTML` or `eval` in src today - keep it that way. Consider httpOnly cookie sessions before scaling.
S12	**SSRF guard on smart import**	OK (resolved)	`server/src/util/scrape.js:6-15`	Already blocks 127/8, 10/8, 172.16-31/12, 192.168/16, 169.254/16, IPv6 loopback.
S13	**File upload validation**	OK	`server/src/routes/uploads.routes.js:24-25` (mime filter) + `server/src/util/images.js` (sharp metadata validates real image) + UUID filenames + path-traversal-safe delete	No action needed.
S14	**Secret exposure** - no API keys in repo	OK	`.env.example` only; CLAUDE_API_KEY blank	Ensure `.env` is in `.gitignore` (already is).
S15	**HTTPS**	Depends on host	Not enforced in app	Use a host that terminates TLS (Render/Railway/Vercel/cPanel all do) + HSTS via helmet.
S16	**Per-tenant data isolation**	N/A	Single-tenant by design	Not applicable.

Net security verdict

Single CRITICAL and 3 HIGH are all **config/middleware additions** - 1-2 days of focused work. The codebase itself is structurally sound (no XSS sinks, no

SQLi surface, parameterised file paths, SSRF guard in place).

PHASE 6 - Performance Audit

Frontend

Metric	Current	Verdict	Action
Bundle size (raw, esbuild)	~600 KB JS + ~3 KB CSS	OK	Vite production build (with minify + gzip + brotli) typically lands ~180-230 KB initial transfer
Code splitting	YES - Vite `manualChunks` for react/redux/firebase/recharts/motion (`vite.config.js`)	GOOD	None
Lazy loading routes	YES - all routes use `React.lazy()` in `src/App.jsx`	GOOD	None
Image optimization	Uploads compressed + WEBP + thumbnails (sharp)	GOOD	Add `srcset` + responsive sizes on product cards
Image lazy loading	`loading="lazy"` on most product/grid images	GOOD	None
PWA	vite-plugin-pwa registered with manifest + workbox precache	GOOD	None
Fonts	Tailwind defaults; no custom font loading bottleneck	OK	None
Third-party drag	firebase (90 KB) declared but unused in core flows	MEDIUM	Remove the firebase dep if you're not using it, OR tree-shake to specific subpackages

Backend

Metric	Current	Verdict	Action
Auth overhead	bcrypt cost 8 (~10 ms) + JWT sign	OK	Raise bcrypt to 12 (acceptable; ~250 ms only at login)
API response (read)	In-memory; sub-ms	EXCELLENT now, fragile at scale	Move to indexed DB
API response (write)	Full JSON file rewrite every call (~5-50 ms for db.json under a few MB)	OK at low traffic, **degrades sharply**	Move to DB
Image processing	sharp + WEBP at upload time	GOOD	Move to background worker at scale

Metric	Current	Verdict	Action
Memory	Whole DB in memory + per-request multer buffers	OK	Use multer disk storage at scale
AI calls	Stub (sync, sub-ms) or live (Claude, 1-3 s)	OK	Add timeout + retry on live mode
Concurrency	Single Node process	OK at low traffic	PM2 cluster or k8s replicas at scale
Caching	None (no Redis)	LOW priority	Introduce Redis at growth stage for sessions + AI response cache

PHASE 7 - Scalability Roadmap

Estimated infrastructure tiers, given the current architecture (with the migrations recommended below). Numbers assume an eCommerce browse-heavy workload (10:1 read/write).

Stage	Users	Servers	Database	Storage	CDN	LB	Notes
Soft launch	0-100	1 (1 vCPU / 1 GB)	JSON file ok	Local disk	Optional	None	Current arch
Stage 1	100-1k	1 (2 vCPU / 2 GB)	Postgres free tier (Supabase)	Local + R2 mirror	Cloudflare in front	None	Migrate persistence
Stage 2	1k-10k	1 (2-4 vCPU / 4 GB)	Postgres $19-25 + 1 read replica	R2 ~5 GB	Cloudflare	None	Add Redis cache
Stage 3	10k-100k	2-3 web (4 vCPU / 8 GB)	Postgres HA $80-150 + replicas	R2 ~50 GB	Cloudflare/ Fastly	YES	Add background worker (BullMQ) for exports / images / AI
Stage 4	100k+	Auto-scaling (k8s) 4+ pods	Postgres clustered + Redis cluster	R2 multi-region (TB)	Multi-edge	YES + WAF	Split into services (orders, catalog, media, exports); event bus

PHASE 8 - Production Deployment Readiness Checklist

Area	Status	Evidence / Action
Infrastructure - single Node service	READY	`server/src/index.js:52-55` serves the built site + API

Area	Status	Evidence / Action
Build pipeline	READY	`npm run build:all`; `nixpacks.toml` for Railway; `app.cjs` for cPanel
Reverse proxy / TLS	READY (via host)	Render/Railway/cPanel terminate TLS automatically
DNS	NOT READY	No domain yet (per user); use host subdomain to launch
CDN	NOT READY	None configured; recommend Cloudflare in front
Database	NOT READY	JSON file - replace with managed Postgres
Storefront data shared across customers	NOT READY (CRITICAL)	Products/blog/orders/reviews are per-browser - must move to backend
Auth & RBAC	READY	JWT + bcrypt + per-route requirePermission + Executive super-admin + impersonation + audit
Audit log	PARTIALLY READY	Capped at 5000 in JSON; move to DB
File uploads	READY	multer + sharp validation + UUID + path-traversal-safe delete
Email (transactional)	NOT READY	mailto-based; needs SMTP (SendGrid/SES/Postmark) or EmailJS for real automated email
Payments	NOT READY	COD + bank-transfer only; no gateway (JazzCash/Easypaisa/Stripe)
Backups	NOT READY	None - depends on host disk snapshot
Disaster recovery	NOT READY	No defined RPO/RTO
Monitoring	NOT READY	Console-only - add an APM (Sentry, Better Stack, New Relic free tier)
Logging	PARTIALLY READY	`console.log`/`console.error`; route through pino + log shipper at growth
CI/CD	PARTIALLY READY	Auto-deploy on Git push when on Render/Railway; no test pipeline
Secrets management	PARTIALLY READY	`.env.example` in repo; need a vault (Doppler, Render env, AWS SM) for prod
Security headers	NOT READY	No helmet middleware
Rate limiting	NOT READY	None
CORS hardening	NOT READY	Wide-open `cors()`

Area	Status	Evidence / Action
Compliance / PII	PARTIALLY READY	Customer data persisted; needs documented retention + DSR (data subject request) workflow
Accessibility	NOT FORMALLY AUDITED	Tailwind defaults are decent; needs WCAG AA pass
SEO foundations	READY	Robots, sitemap, manifest, FAQ/Article/Product/Breadcrumb schema, AI optimisation panel

Readiness Score

Bucket	Score
Build & dev experience	9/10
Frontend quality	8/10
Backend code quality	7/10
Auth + RBAC	9/10
Data layer	3/10 (JSON file + browser localStorage)
Security baseline	5/10 (defaults need overriding; no headers / rate limit / hardened CORS)
Observability	2/10
Backups / DR	1/10
Payments	2/10
Email infra	2/10
Scalability runway	5/10
Weighted overall	**58/100 - soft-launch grade**

PHASE 9 - Hosting Cost Analysis (USD / month)

All figures are conservative public-list prices; actual bills vary with region and reserved-instance pricing.

Startup stage (0-1,000 users)

Item	Provider	Monthly
App hosting (single Node service)	Render free / Railway free credit / cPanel basic	$0-5
Database	Supabase free tier / Neon free tier	$0
Object storage	Cloudflare R2 (<10 GB)	<$1

Item	Provider	Monthly
CDN	Cloudflare free	$0
Email (1k mails/mo)	SendGrid free / Resend free	$0
Monitoring	Sentry / Better Stack free	$0
Backups	Manual / host snapshot	$0
Domain	Namecheap .pk	~$5/yr (=~$0.40/mo)
Total		**$0-6**

Growth stage (1,000-10,000 users)

Item	Provider	Monthly
App hosting (1 always-on instance, 2 vCPU/4 GB)	Render Starter ($7-25), Railway ~$10-25, DO App $12, VPS $5-15	$10-25
Database	Supabase Pro ($25) / Neon Launch ($19) / DO Managed PG ($15)	$19-25
Object storage (~5-20 GB)	Cloudflare R2	$1-3
CDN	Cloudflare free or Pro ($20)	$0-20
Email (10-50k mails)	SendGrid Essentials ($20) / Resend Pro ($20)	$20
Monitoring	Sentry Team ($26)	$0-26
Backups	DB built-in (managed)	$0
Domain + SSL	host included	$0
Total		**$50-119**

Scale stage (10k-100k users)

Item	Provider	Monthly
App hosting (2-3 instances, auto-scale)	Render Pro / Railway / Fly	$80-200
Database (HA, read replica)	Supabase Team ($599) / RDS db.t4g.small + replica (~$60-100)	$60-600
Redis cache	Upstash / Redis Cloud	$10-30
Object storage (~50-200 GB)	R2	$1-5
CDN	Cloudflare Pro ($20) or Fastly	$20-80
Email (~250k mails)	SendGrid Pro ($90)	$90
Monitoring + log mgmt	Sentry Business + Better Stack	$80-150
Background workers	Same host or Render Cron	$20-50
Total		**$361-1,205**

Enterprise stage (100k+ users)

Item	Monthly
Multi-region web tier (k8s/Fargate)	$500-2,000
Postgres clustered + replicas	$500-3,000
Redis cluster	$100-400
Object storage + multi-region replication	$50-300
CDN + WAF (Cloudflare Business/Enterprise)	$200-2,000
Email at scale (1M+ mails)	$200-800
Observability (Datadog/New Relic)	$300-1,500
Backups + DR + cold storage	$100-300
Total	**$1,950-10,300**

PHASE 10 - Final CTO Report

1. Executive Summary

Al Rafiq Shopping Centre is a clean, comprehensive, feature-rich eCommerce codebase wrapped around a single Node service. The code itself is solid: component structure is sane, the RBAC model is real (role-based + super-admin + impersonation + audit log), the SEO/AI features work, and the build pipeline is reliable. **However, two structural choices are blockers for a real revenue storefront**: the JSON-file datastore and the localStorage-only storefront. A focused 2-3 week migration to managed Postgres (Supabase) + a backend-served storefront would convert this from "great staging build" to "true production storefront".

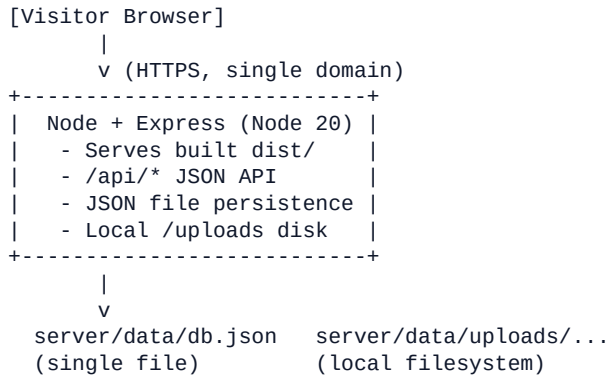
2. Current Technical Health Score

Dimension	Score
Code quality	8.0
Architecture	5.5 (single-node JSON store)
Security baseline	5.5 (config-level fixes only)
Performance	7.5 (good frontend; backend is fast at low scale)
Scalability	4.0 (architectural ceiling)
Observability	2.0
Operability	6.0

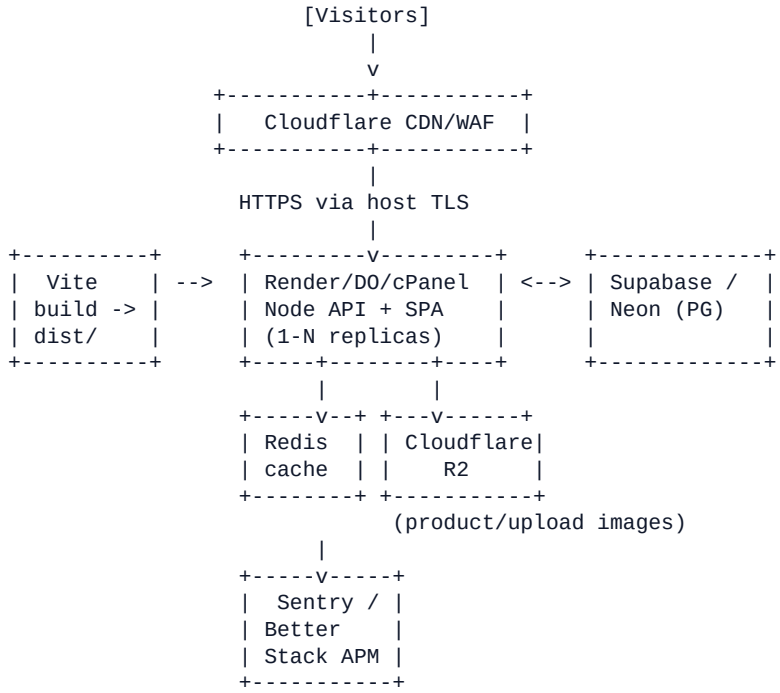
Dimension	Score
Overall	**58/100**

3. Infrastructure Architecture Diagram (text)

Today (single service)



Recommended (production target)



4. Hosting Recommendations (by horizon)

Horizon	Recommendation
Now (testing)	**Railway free** OR **cPanel Node.js App** with persistent disk
0-1k users	**Render Starter $7-25/mo** OR DO App Platform $12/mo, with **Supabase free**
1-10k users	Render Standard + Supabase Pro $25 + Cloudflare free

Horizon	Recommendation
10-100k users	Scale-tier Render/Fly + Supabase Team / RDS HA + Cloudflare Pro + Redis
100k+	k8s + clustered DB + multi-region

5. Database Recommendation

Supabase (managed PostgreSQL) for primary persistence. Why this fit:

- Cleanly replaces the JSON store and the per-browser Redux persistence.
- Includes Row-Level Security that mirrors the existing `requirePermission` RBAC and the executive impersonation pattern.
- Bundles object storage to replace `server/data/uploads/`.
- Free tier covers soft launch; predictable jump to $25 at growth.

Schema sketch (1:1 with current Redux slices):
`users`, `admin_users`, `permissions_audit`, `products`, `categories`, `tags`, `product_tags`, `orders`, `order_items`, `customers`, `reviews`, `complaints`, `complaint_events`, `complaint_notes`, `blog_posts`, `blog_faqs`, `accounts_txn`, `media_library`, `audit_log`, `export_log`, `import_log`, `settings`.

6. Security Recommendations (prioritised, P0-P3)

Priority	Item
P0	Refuse boot if `JWT_SECRET` is default/missing in prod; force-reset seeded passwords on first login.
P0	Add `helmet()`, `express-rate-limit` (esp. on `/api/auth/login` and `/api/ai/generate/*`), tighten `cors()` to your FRONTEND_URL.
P0	Reduce JSON body limit to 1 MB; keep multer's 12 MB only on the multipart upload route.
P1	Raise bcrypt cost to 10-12.
P1	Move audit log out of the JSON cap into a real DB table; define retention.
P1	Move JWT to httpOnly cookies once you split frontend/backend domains.
P2	Add CSP, HSTS, Referrer-Policy.
P2	Document privacy/DSR workflow; add unsubscribe + consent capture for email.
P3	Penetration test before public launch.

7. Cost Analysis (summary)

Stage	All-in monthly
Soft launch	**$0-6**

Stage	All-in monthly
Growth (1-10k)	**$50-119**
Scale (10-100k)	**$361-1,205**
Enterprise	**$1,950-10,300**

8. Scalability Roadmap (4 milestones)

1. **MVP live (week 1)**: Render/Railway/cPanel single service, current arch, internal data only.

2. **Real store (weeks 2-4)**: migrate products/blog/orders/reviews to Postgres; flip storefront to fetch from backend; add payment gateway (JazzCash/Easypaisa/Stripe) and SMTP email.

3. **Growth (month 2-6)**: add Redis cache, R2 for media, CDN, Sentry, basic CI/CD with tests, helmet + rate limit + tight CORS.

4. **Scale (>10k MAU)**: horizontal web replicas behind LB, background workers for exports/AI/images, Postgres HA + read replica.

9. Launch Risks (top 10)

#	Risk	Likelihood	Impact	Mitigation
1	Products you add in admin aren't visible to other customers	**CERTAIN today**	High	Backend data migration
2	Default JWT secret stays in prod	High	Critical	Refuse-boot guard
3	Default seeded passwords reused	High	Critical	Force reset on first login
4	Brute-force on `/api/auth/login`	Medium	High	Rate limit + lockout
5	CORS wide-open lets any site call your API	Medium	High	Tighten origin
6	JSON-file write races corrupt data	Low at low load, High at scale	Critical	Move to Postgres
7	Free-tier host sleeps - first visit slow	High	Low	Upgrade $7 to keep awake
8	Disk full or db.json grows unbounded	Medium	High	Migrate persistence + monitoring
9	No payment gateway = manual COD only	Certain	Medium	Integrate JazzCash/ Easypaisa/Stripe
10	No backups	Certain	Critical	Managed DB + scheduled R2 snapshots

10. Final Go / No-Go

Scenario	Verdict
Internal demo / staging / pitch / friends-and-family beta	**GO** - deploy today as-is
Real public storefront with paying customers	**NO-GO** until P0 security fixes + storefront data migrated to backend + at least one online payment method
High-traffic launch / marketing push	**NO-GO** until Stage-2 architecture (Postgres + Redis + CDN + monitoring)

Minimum acceptance criteria to flip to GO for a real store

1. JSON store replaced by managed Postgres (Supabase).
2. Storefront reads products/blog/orders/reviews from backend (no per-browser data for shared entities).
3. `JWT_SECRET`, `SITE_URL`, `FRONTEND_URL` set; default-secret refusal on boot.
4. Default seeded passwords reset; forced password change on first login.
5. `helmet()`, `express-rate-limit`, hardened `cors()` middleware live.
6. Body limit reduced; bcrypt cost raised to 12.
7. One real payment method integrated.
8. Real transactional email (SMTP / SendGrid / Resend).
9. Basic monitoring (Sentry) + DB daily backup.
10. Run an `npm audit` and pin major versions.

Audit produced from direct inspection of the repo; every finding above cites a file or behavior in the codebase. Re-run after each major change.